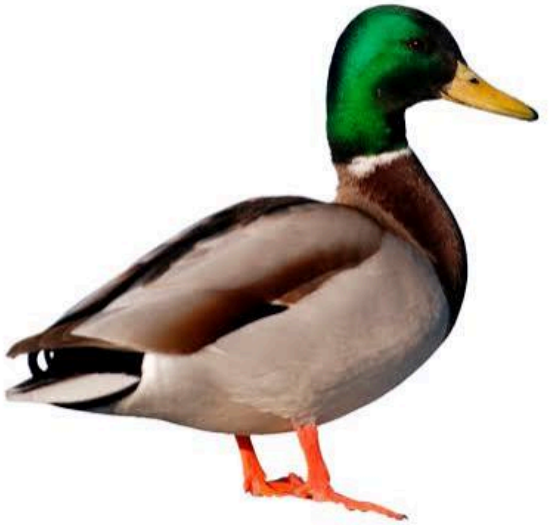
Ciência de Dados Aplicada ao Direito I

Aula 11

Notebook, print, comentário, type() e os primeiros erros



Prof. Bruno da Cunha de Oliveira - 2026/2

Dois jeitos de rodar Python

Abrir a pasta cdad1, sempre

No VS Code, File > Open Folder e escolher a `cdad1`. Não o arquivo, não a pasta de cima: a `cdad1`, que tem a `.venv` dentro.

É dentro da pasta aberta que o VS Code procura o Python do projeto. Aberta outra pasta, ele não acha a `.venv`, e o notebook roda num Python que não tem o `ipykernel`.

Certo

```
cdad1  (.venv)
```

O seletor no canto do notebook diz `.venv`.

Errado

```
Python 3.10.12
```

Um Python global, sem os pacotes do projeto. Trocar pelo seletor, ou fechar e abrir a `cdad1`.

ola.py: o primeiro programa

Um arquivo novo na pasta, chamado `ola.py`, com uma linha:

```
print("Olá, turma!")
```

E, no terminal, dentro de `cdad1`:

```
uv run ola.py
```

Se apareceu Olá, turma! no terminal, o ambiente está pronto.

Demonstração: rodar o ola.py

O mesmo, num notebook

Um arquivo novo, `ola.ipynb` (de IPython Notebook IPython é “Interactive Python”). O VS Code abre uma célula: uma caixa em que se escreve código.

```
print("Olá, turma!")
```

Shift + Enter roda a célula, e o resultado aparece logo abaixo dela. Na primeira vez, o VS Code pergunta qual Python usar: é o da `.venv` do projeto.

Demonstração: a primeira célula

Insper

print, ou só o nome?

No notebook, a última linha da célula aparece embaixo sozinha, sem `print`. É o notebook devolvendo o resultado da última expressão, e só dela.

Só o nome

```
prazo = 15
prazo
```

```
15
```

Com print

```
prazo = 15
print(prazo)
```

```
15
```

Duas expressões: só a última

```
prazo
prazo * 2
```

```
30
```

Num `.py`, `prazo` sozinho não mostra nada. Só o `print` escreve na tela, e é ele que funciona nos dois.

Por que o notebook existe

- No `.py`, o programa roda inteiro, do começo ao fim, toda vez. Para mudar uma linha, roda tudo de novo.

Por que o notebook existe

- No `.py`, o programa roda inteiro, do começo ao fim, toda vez. Para mudar uma linha, roda tudo de novo.

- No notebook, cada célula roda separada, e o que ela produziu fica na memória para a célula seguinte.

Por que o notebook existe

- No `.py`, o programa roda inteiro, do começo ao fim, toda vez. Para mudar uma linha, roda tudo de novo.
- No notebook, cada célula roda separada, e o que ela produziu fica na memória para a célula seguinte.
- Com uma base de dados grande, isso é a diferença entre esperar dois minutos a cada tentativa e esperar uma vez só: carrega numa célula, explora nas outras.

Por que o notebook existe

- No `.py`, o programa roda inteiro, do começo ao fim, toda vez. Para mudar uma linha, roda tudo de novo.
- No notebook, cada célula roda separada, e o que ela produziu fica na memória para a célula seguinte.
- Com uma base de dados grande, isso é a diferença entre esperar dois minutos a cada tentativa e esperar uma vez só: carrega numa célula, explora nas outras.
- O resultado fica ao lado do código que o produziu: texto, tabela, gráfico. É por isso que ciência de dados vive em notebook.

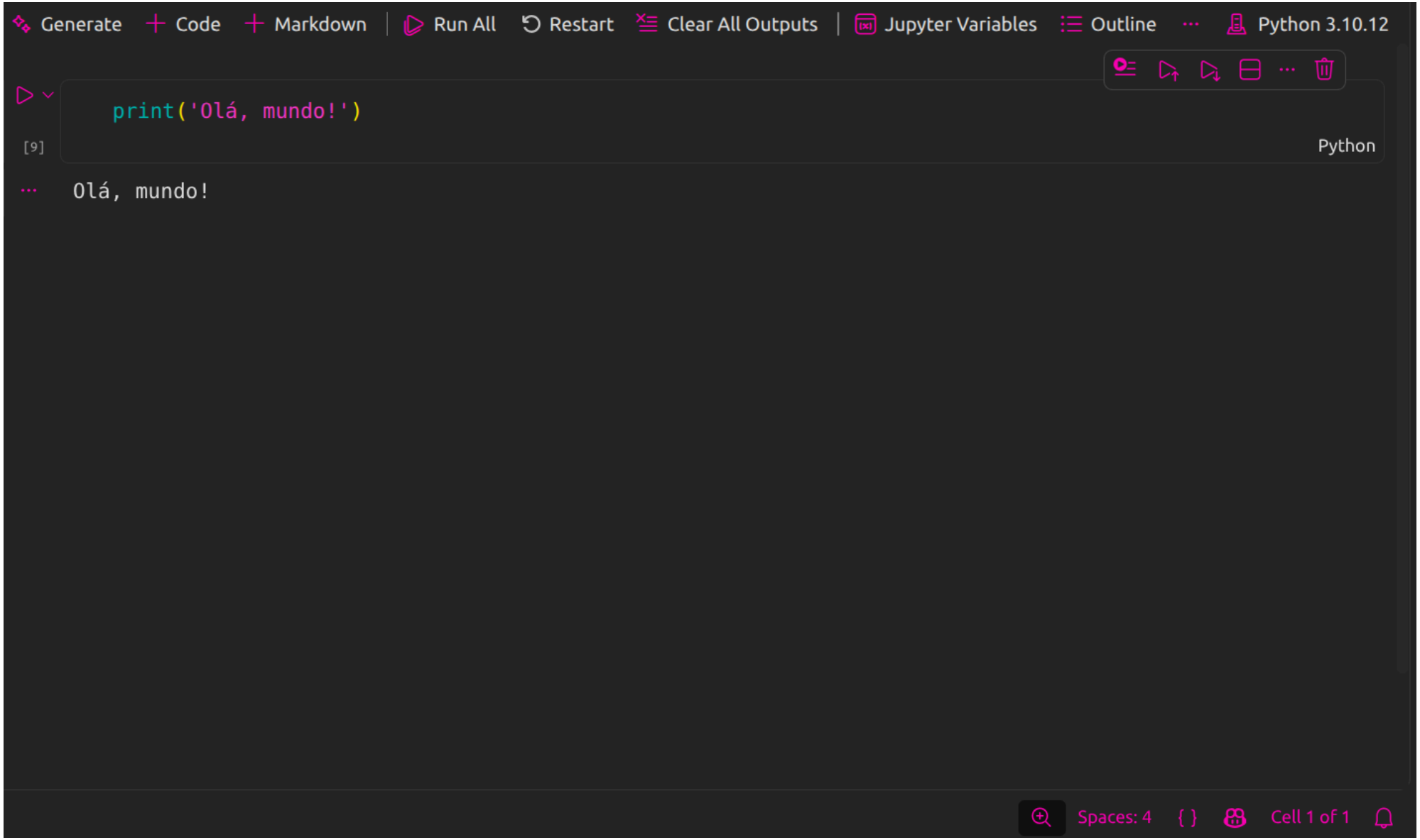
Quando usar cada um

`.py`	`.ipynb`
Um programa que roda inteiro e faz uma coisa.	Explorar dados, tentar, olhar, tentar de novo.
Roda pelo terminal: `uv run arquivo.py`.	Roda célula a célula, dentro do VS Code.
O resultado vai para o terminal.	O resultado fica embaixo da célula, e é salvo com o arquivo.
É o que a gente entrega como ferramenta.	É onde a gente pensa.

Neste curso, o trabalho é sempre em `.ipynb`. O `.py` volta em alguns casos de vibecoding, mais adiante.

A interface do notebook

Um notebook no VS Code, com uma célula rodada



A barra de cima

Botão	O que faz
`+ Code`	Célula nova de código: Python, que roda.
`+ Markdown`	Célula nova de texto: título, parágrafo, lista. Não roda; é o enunciado, a explicação.
`Run All`	Roda todas as células, de cima para baixo.
`Restart`	Reinicia o Python: apaga tudo que estava na memória.
`Clear All Outputs`	Limpa os resultados de baixo das células. A memória continua.
`Python 3.10.12`, no canto	O seletor do Python. Tem de dizer `.venv`.

Code e Markdown

Célula de código

```
prazo = 15
print(prazo * 2)
```

Python. Shift + Enter roda, e o resultado aparece embaixo.

Célula de Markdown

```
## 2. Escrever prints

Escreva o código que produz
**exatamente** a saída pedida.
```

Texto. `#` faz título, `**` faz negrito. Shift + Enter só formata.

Num notebook de exercícios, o enunciado é Markdown e a resposta é código. Escrever Python numa célula de Markdown não dá erro: não acontece nada, e é por isso que confunde.

A memória do notebook

O que uma célula criou fica na memória, e a ordem que vale é a de execução, não a de cima para baixo.

Célula de cima

```
prazo = 15
```

Célula de baixo

```
print(prazo * 2)
```

Rodou a de baixo antes da de cima

```
NameError: name 'prazo' is not defined
```

O Python nunca viu `prazo`. Rodar a de cima e depois a de baixo resolve.

`Restart` apaga a memória inteira. Depois dele, `Run All`: se o notebook roda de cima a baixo sem erro, ele está certo. Se não roda, alguma célula dependia de outra que já não existe.

Comentário

Comentário

Em Python: `# texto`

Tudo que vem depois de `#` na linha, o Python ignora. É escrito para quem lê o código: você daqui a um mês, o colega do grupo, a IA.

EXEMPLO

```
# a Defensoria tem prazo contado em dobro (art. 186 do CPC)
prazo = 15 * 2

print(prazo)
```

O que vale a pena comentar

- Por que, e não o quê. `prazo = 15 * 2` já mostra a conta; o comentário diz de onde vem o dobro, que é o que ninguém adivinha lendo o código.

O que vale a pena comentar

- Por que, e não o quê. `prazo = 15 * 2` já mostra a conta; o comentário diz de onde vem o dobro, que é o que ninguém adivinha lendo o código.

- Uma decisão que não é óbvia, um número que veio de algum lugar, um cuidado que a pessoa seguinte vai querer tirar.

O que vale a pena comentar

- Por que, e não o quê. `prazo = 15 * 2` já mostra a conta; o comentário diz de onde vem o dobro, que é o que ninguém adivinha lendo o código.

- Uma decisão que não é óbvia, um número que veio de algum lugar, um cuidado que a pessoa seguinte vai querer tirar.

- Comentário também serve para desligar uma linha sem apagar: põe `#` na frente, e ela deixa de rodar.

print

print é uma função

Uma função tem nome, recebe entradas entre parênteses e devolve uma saída, como o `prazo_final(15, VERDADEIRO)` do pseudocódigo. `print` é uma função que o Python já traz pronta.

```
print("Prazo:", prazo)
```

`print` é o nome; os parênteses fazem a chamada o que vai dentro são os argumentos, quantos forem, separados por vírgula.

E ela não devolve nada: só mostra. Por isso o notebook exibe `prazo` sozinho, que é um valor devolvido, e `print(prazo)` escreve na tela sem devolver.

print: mostrar na tela

Em Python: `print(...)`

O que estiver entre os parênteses aparece na tela. É o único jeito de o programa contar o que fez, e por isso é a primeira coisa que se aprende.

EXEMPLO

```
print("Olá, turma!")
print(15 * 2)

prazo = 15
print(prazo)
```

Ler código sem rodar

De vez em quando eu escrevo uma célula, pergunto o que vai aparecer na tela, ouço os palpites, e só então rodo.

O treino é ler um código e saber o que ele faz sem abrir o VS Code e rodar linha por linha. É o que vocês vão fazer o tempo todo com código escrito por um colega ou por uma IA: ler, entender, decidir se está certo.

Quem só sabe rodar depende de rodar. Quem lê, confere.

O que aparece na tela?

```
print("Olá, turma!")
print(15 * 2)

prazo = 15
print(prazo)
print("prazo")
```

O que aparece na tela?

RESPOSTA

```
Olá, turma!
30
15
prazo
```

`print(prazo)` mostra o valor da variável. `print("prazo")` mostra o texto, porque as aspas fazem dele um texto. É a diferença entre o nome e o texto.

Aspas simples ou duplas, e a mesma para fechar

Texto vai entre aspas, e o Python aceita as duas: `'Olá'` e `"Olá"` são o mesmo texto. A regra é uma só: fecha com a mesma aspa que abriu.

Funciona

```
print('Olá, mundo!')
print("Olá, mundo!")
```

Erro: abriu com uma, fechou com outra

```
print('Olá, mundo!")
```

Para que a outra serve: quando o texto tem uma aspa dentro.

```
print("O art. 5º diz 'todos são iguais'.")
```

Vários de uma vez: a vírgula

Dentro do `print`, a vírgula separa os argumentos, e o Python põe um espaço entre cada um. Serve para misturar texto e valor:

```
prazo = 15
print("Prazo:", prazo, "dias")
```

```
Prazo: 15 dias
```

`print("Prazo: prazo")` mostraria a palavra `prazo`, e não `15`: dentro das aspas é texto, e o valor só entra como argumento separado.

Quebra de linha dentro do texto: \n

`\n` é a quebra de linha: dois caracteres na tela do editor, e uma linha nova quando o texto é mostrado.

```
print("Prazo:\n15 dias")
```

```
Prazo:
15 dias
```

O `n` é de “new line”. A barra avisa que o `n` não é a letra, e sim um comando.

Linha em branco: print() vazio

Vários `print`, uma linha cada

```
print("Art. 1º")
print("Art. 2º")
```

```
Art. 1º
Art. 2º
```

`print()` sem nada pula uma linha

```
print("Art. 1º")
print()
print("Art. 2º")
```

```
Art. 1º

Art. 2º
```

Cada `print` termina a linha sozinho. Um `print()` vazio termina uma linha em que não havia nada: é a linha em branco.

PrairieLearn

Insper

O PrairieLearn: onde os exercícios são corrigidos

Os exercícios de sala são notebooks que rodam no navegador, dentro do PrairieLearn, e um corretor automático confere a resposta na hora.

1. Cada questão tem uma célula de resposta, e a primeira linha dela é `#grade`. Ela não se apaga: é por ela que o corretor acha o código.
2. Os nomes das variáveis são exatamente os do enunciado. O corretor procura por nome.
3. Salvar, e depois corrigir. Errou? Conserta e corrige de novo, quantas vezes precisar.

O VS Code fica aberto do lado, para testar. A resposta que vale é a do PrairieLearn.

Cadastro e login, agora

O link do curso está no Blackboard. Todo mundo entra agora, antes do primeiro exercício:

1. Abrir o link e entrar com o e-mail do Insper.
2. Conferir que o curso aparece na lista, e abrir a primeira avaliação.
3. Abrir o notebook e esperar o ambiente carregar: a primeira vez demora um pouco.

Quem não conseguir entrar levanta a mão. Ninguém começa o exercício antes de todo mundo estar dentro.

No PrairieLearn: exercicio1

Insper

Quando o Python reclama

Como se lê uma reclamação

Quando alguma coisa não roda, o Python escreve um texto vermelho. Ele se lê de baixo para cima:

```
NameError                                 Traceback (most recent call last)
Cell In[3], line 2
      1 comarca = "São Paulo"
----> 2 print(Comarca)

NameError: name 'Comarca' is not defined
```

1. A última linha diz o tipo do erro, antes dos dois pontos, e a mensagem, depois.
2. A seta `---->` e o `line 2` dizem onde ele aconteceu.
3. O que está acima disso é o caminho até lá, e por enquanto pode ser ignorado.

SyntaxError: o Python não conseguiu ler

Erro de sintaxe: o texto não é Python. Nada roda, nem a linha anterior.

A aspa que não fechou

```
print("Petição inicial')
```

```
  Cell In[1], line 1
    print("Petição inicial')
          ^
SyntaxError: unterminated string
literal (detected at line 1)
```

O comentário sem #

```
prazo = 15 * 2 prazo em dobro
```

```
  Cell In[2], line 1
    prazo = 15 * 2 prazo em dobro
                   ^
SyntaxError: invalid syntax
```

A mensagem nem sempre diz o que faltou. O ^ diz onde o Python se perdeu, e o erro costuma estar ali ou logo antes: uma aspa, um parêntese, um #.

NameError: esse nome não existe

Erro de execução: o Python leu, começou a rodar, e pediu um nome que nunca foi criado.

```
comarca = "São Paulo"
print(Comarca)
```

```
NameError: name 'Comarca' is not defined
```

Três causas, sempre as mesmas: letra trocada (`Comarca` e `comarca` são nomes diferentes), a variável ainda não foi criada (a célula que a cria não rodou), ou faltaram as aspas num texto, e o Python o leu como nome.

O que fazer com cada um

Última linha	O que aconteceu	Onde olhar
`SyntaxError`	O texto não é Python.	O ^ e o que vem logo antes dele: aspa, parêntese, #, dois pontos.
`NameError`	Um nome que não existe.	A grafia do nome; se a célula que o cria já rodou; se faltou aspa num texto.

Em todo caso: ler a última linha, achar a linha apontada, consertar, rodar de novo. Errar e consertar é o jeito normal de programar, e não a exceção.